

Iterating the Scatman idea with Braille

The image displays a 10x10 grid of 100 small square plots. Each plot contains a unique arrangement of black dots on a white background. The dots are distributed in various patterns, including clusters, lines, and scattered points, illustrating different spatial configurations or distributions. The patterns vary significantly from one plot to another, suggesting a systematic exploration of dot arrangements.

```
let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',  
            'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']
```

```
// sandbox - start
```

```
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy
```

```
s0.initP5() // send p5 to hydra
```

```
P5.toggle(0) // optionally hide p5
```

src(s0)

```
.rotate( () => mouse.x * 0.005 )
.pixelate( () => mouse.y, () => mouse.y )
.out()
```

```
// sandbox – end
```

[illegible]

```

createCanvas(windowWidth, windowHeight);
words = lyrics.split(" ");
textSize(40);
textAlign(CENTER, CENTER);
colorBG = color(255);
colorText = color(0);
interval = 80;

setupAudio(true) // global vars
// a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  background(colorBG)

  // Nächstes Wort hinzufügen, wenn Zeit abgelaufen
  if(millis() - lastTime > interval && shownWords.length < words.length) {
    shownWords.push(words[shownWords.length]);
    lastTime = millis();
  }

  // Aktuellen Text zeichnen
  fill(colorText);
  let currentText = shownWords.join(" ");
  text(currentText, width / 2, height / 2);
}

function mousePressed() {
  // Neustart bei Klick
  song.stop()
  song.play()

  shownWords = [];
  lastTime = millis();
  if(mouseIsPressed) {
    if(mouseX < width / 2) {
      colorText = color(255, 0, random(255));
    } else {
      colorBG = color(random(255), 255, 0);
    }
  }
  interval = random(1, 80)
}

```

```
//text to speech
//blindensprache für Scatman per sound

/*
P5LIVE – Audio
If using outside P5LIVE, include p5live-audio.js
https://cdn.jsdelivr.net/gh/ffd8/P5LIVE/includes/utils/p5live-audio.js
*/
```

Pulsing Scatman

ba-da-ba-be bop-bop-bodda
Bop-ba-bodda-bope
p-ba-bodda-bope, bop-ba-l
-ba-da-ba-be bop-ba-bodda
Bop-ba-bodda-bope
ba-bodda-bope, bop-ba-bod
ody stutters, one way or th
check out my message to y
of fact, I don't let nothin' ho

```
// {"P5LIVE":{"name":"Scatman_Pulsierend","mod":1780345210688}}
```

```
let song;
let amp, fft;

let lyrics = `Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub (I'm the Scatman)
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ba-da-ba-da-ba-be bop-bop-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda
Ba-da-ba-da-ba-be bop-ba-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda-bope
Everybody stutters, one way or the other
So check out my message to you
As a matter of fact, I don't let nothin' hold you back
If the Scatman can do it, so can you
Everybody's sayin' that the Scatman stutters
But doesn't ever stutter when he sings
But what you don't know, I'm gonna tell you right now
That the stutter and the scat is the same thing, yo
I'm the Scatman
Where's the Scatman?
I'm the Scatman
Why should we be pleasing in the politician heathens
Who would try to change the seasons if they could?
The state of the condition insults my intuitions
And it only makes me crazy and a heart like wood
Everybody stutters one way or the other
So check out my message to you
As a matter of fact, I'm letting nothing hold you back
If the Scatman can do it, brother, so can you
I'm the Scatman
Ba-da-ba-da-ba-be bop-bop-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda
Ba-da-ba-da-ba-be bop-ba-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda-bope
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Everybody stutters one way or the other
So check out my message to you
As a matter of fact, I don't let nothing hold you back
```

If the Scatman can do it, so can you
I hear you all ask 'bout the meaning of scat
Well, I'm the professor and all I can tell you is
While you're still sleeping, the saints are still weeping 'cause
Things you called dead haven't yet had the chance to be born
I'm the Scatman
Ba-da-ba-da-ba-be bop-bop-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda
Ba-da-ba-da-ba-be bop-ba-bodda-bope
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda-bope
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Ski-bi-dibby-dib yo-da-dub-dub, yo-da-dub-dub
Yeah, I'm the Scatman
Dong dong dong, ding ding-dong, dong
Wo-go-ez-ze-ze-za-de-ya-de-za-de-ya-ze-ze-zee
Dong-ding-dong-dong-dong, do-dong-ding
Where's the Scatman?
I'm the Scatman, repeat after me
It's a scoobie-oobie-doobie, scoobie-doobie melody
I'm the Scatman, sing along with me
It's a scoobie-oobie-doobie, scoobie-doobie melody
Yeah, I'm the Scatman (I'm the Scatman)
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope, bop-ba-bodda
I'm the Scatman (I'm the Scatman)
Bop-ba-bodda-bope
Be-bop-ba-bodda-bope`;
`let words = [];`
`let shownWords = [];`
`let lastTime = 0;`
`let interval = 80;`
`let bgColor, textColor;`

`function preload() {`
 `song = loadSound('scatman.mp3');`
`}`

`function setup() {`
 `createCanvas(windowWidth, windowHeight);`
 `words = lyrics.split(" ");`
 `textAlign(CENTER, CENTER);`

```

amp = new p5.Amplitude();
fft = new p5.FFT();

bgColor = color(0);
textColor = color(255);
}

function draw() {
  // Audio analysieren
  fft.analyze();
  let level = amp.getLevel();
  let bass = fft.getEnergy("bass");
  let treble = fft.getEnergy("treble");

  // Hintergrund pulsiert mit Bass
  background(bass * 0.5, 0, bass * 0.3);

  // Textgröße pulsiert mit Lautstärke
  let size = map(level, 0, 0.4, 30, 120);
  textSize(size);

  // Geschwindigkeit hängt von Höhen ab → höhere Töne = schnellere Wörter
  let dynamicInterval = map(treble, 0, 255, 1000, 100);

  if (millis() - lastTime > dynamicInterval && shownWords.length <
words.length) {
    shownWords.push(words[shownWords.length]);
    lastTime = millis();
  }

  // Textfarbe ändert sich mit Bass
  fill(255 - bass, 255, bass);
  text(shownWords.join(" "), width/2, height/2);
}

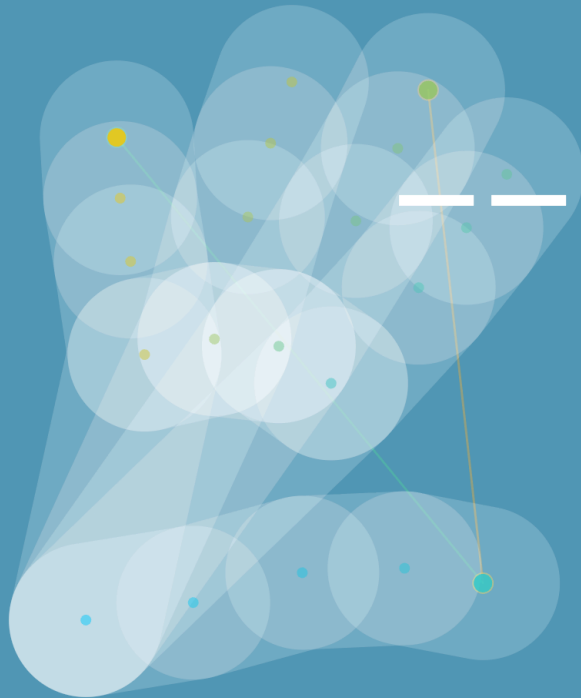
function mousePressed() {
  song.stop()
  song.play()

  shownWords = [];
  lastTime = millis();
}

```


Trying out new stuff by including hand gestures with morse code

👉 Daumen + kleiner Finger → · (Punkt)
👉 Daumen + Mittelfinger → – (Strich)
🕒 Pause 1.5s → Buchstabe
= . – Space (Fallback)



m

[. . . .]

```
// {"P5LIVE":{"name":"Hand Animation","mod":1780345728389}}

// {"P5LIVE":{"name":"Morse HandPose Control","mod":1779614887334}}
// _____
// STEUERUNG:
// 👉 Daumen + kleiner Finger zusammen → · (Punkt)
// 👉 Daumen + Mittelfinger zusammen → – (Strich)
// 🕒 1.5 Sek. Pause → Buchstabe dekodieren
//
// Tastatur-Fallback:
// . → Punkt – → Strich Space → jetzt dekodieren
// Backspace → letztes Symbol / Buchstaben löschen
// _____
```

```

let libs = ['https://unpkg.com/ml5@1/dist/ml5.min.js'];

// — Morse —————
let bgColor;
let typedText = '';
let morseItems = [];
let synth;
const morseSize = 2;

const morse = {
  a: ".-.", b: "-...", c: "-.-.", d: "-..",
  e: ".", f: "..-", g: "--.", h: "....",
  i: "..", j: ".---", k: "-.-", l: "-..",
  m: "--", n: "-.", o: "---", p: "--.",
  q: "--.-", r: ".-.", s: "...", t: "-",
  u: "..-", v: "...-", w: ".---", x: "-.-.",
  y: "-.-.", z: "--.."
};

const morseReverse = {};
for (let l in morse) morseReverse[morse[l]] = l;

// — HandPose —————
let handPose;
let video;
let hands = [];
const handOptions = { maxHands: 1, flipHorizontal: true };

// — Gestensteuerung —————
let morseBuffer = '';
let lastInputTime = 0;
const LETTER_TIMEOUT = 1500; // ms Pause → Buchstabe
const PINCH_THRESHOLD = 0.28; // normierter Abstand → Pinch aktiv

// Zustandsmaschine: löst nur beim ersten Kontakt aus, nicht bei Halten
let dotWasPinching = false; // Daumen + kleiner Finger
let dashWasPinching = false; // Daumen + Mittelfinger

// Geglättete Abstände für Visualisierung
let smoothDotDist = 1;
let smoothDashDist = 1;
const DIST_SMOOTH = 0.25;

// — MediaPipe Hand-Landmarks —————
// 0 = Handgelenk
// 4 = Daumen-Spitze

```

```

// 8 = Zeigefinger-Spitze
// 12 = Mittelfinger-Spitze
// 16 = Ringfinger-Spitze
// 20 = Kleiner-Finger-Spitze
// 9 = Mittelfinger-MCP (für Handgrösse)

// — Setup —————
function preload() {
  handPose = ml5.handPose(handOptions);
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  bgColor = color(255, 0, 127);

  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);
  textFont('monospace');

  video = createCapture(VIDEO, { flipped: true });
  video.size(width, height);
  video.hide();

  handPose.detectStart(video, gotHands);
}

// — Draw —————
function draw() {
  background(bgColor);

  // Auto-Dekodierung nach Timeout
  if (morseBuffer.length > 0 && millis() - lastInputTime > LETTER_TIMEOUT) {
    decodeLetter();
  }

  // Hand-Analyse
  if (hands.length > 0) {
    detectPinches(hands[0]);
    drawHandSkeleton(hands[0]);
  }

  // Schwebende Morsezeichen
  for (let item of morseItems) {
    item.targetX += random(-0.3, 0.3);
    item.targetY += random(-0.3, 0.3);
  }
}

```

```

    for (let other of morseItems) {
      if (item !== other) {
        let d = dist(item.x, item.y, other.x, other.y);
        if (d < 170) {
          let angle = atan2(item.y - other.y, item.x - other.x);
          let force = (170 - d) * 0.05;
          item.targetX += cos(angle) * force;
          item.targetY += sin(angle) * force;
        }
      }
    }
    item.targetX = constrain(item.targetX, 100, width - 100);
    item.targetY = constrain(item.targetY, 100, height - 200);
    item.x = lerp(item.x, item.targetX, 0.03);
    item.y = lerp(item.y, item.targetY, 0.03);
    drawMorse(item);
  }

  // Getippter Text
  fill(255);
  noStroke();
  textAlign(CENTER);
  textSize(40);
  text(typedText, width / 2, height - 80);

  // Puffer + Fortschrittsbalken
  let progress = morseBuffer.length > 0
    ? map(millis() - lastInputTime, 0, LETTER_TIMEOUT, 0, 1, true) : 0;
  drawBuffer(progress);

  drawHUD();
}

// — Pinch-Erkennung —————
function detectPinches(hand) {
  let kp = hand.keypoints;

  // Handgrösse: Handgelenk → Mittelfinger-MCP
  let scale = dist(kp[0].x, kp[0].y, kp[9].x, kp[9].y);
  if (scale === 0) return;

  let thumb = kp[4]; // Daumen-Spitze
  let pinky = kp[20]; // Kleiner-Finger-Spitze
  let middle = kp[12]; // Mittelfinger-Spitze

  let dotDist = dist(thumb.x, thumb.y, pinky.x, pinky.y) / scale;

```

```

let dashDist = dist(thumb.x, thumb.y, middle.x, middle.y) / scale;

// Smooth für Visualisierung
smoothDotDist = lerp(smoothDotDist, dotDist, DIST_SMOOTH);
smoothDashDist = lerp(smoothDashDist, dashDist, DIST_SMOOTH);

// DOT: Daumen + kleiner Finger
if (dotDist < PINCH_THRESHOLD) {
  if (!dotWasPinching) {
    addSymbol(".");
    dotWasPinching = true;
  }
} else {
  dotWasPinching = false;
}

// DASH: Daumen + Mittelfinger
if (dashDist < PINCH_THRESHOLD) {
  if (!dashWasPinching) {
    addSymbol("-");
    dashWasPinching = true;
  }
} else {
  dashWasPinching = false;
}

// Visuelles Feedback zwischen den Fingern
drawPinchLine(thumb, pinky, smoothDotDist, ".", color(80, 220, 160));
drawPinchLine(thumb, middle, smoothDashDist, "-", color(255, 180, 60));
}

function drawPinchLine(a, b, normDist, label, col) {
  let active = normDist < PINCH_THRESHOLD;
  let alpha = active ? 255 : 100;

  // Linie zwischen den Fingerspitzen
  stroke(red(col), green(col), blue(col), alpha);
  strokeWeight(active ? 3 : 1.5);
  line(a.x, a.y, b.x, b.y);

  // Punkte auf den Spitzen
  noStroke();
  fill(red(col), green(col), blue(col), alpha);
  circle(a.x, a.y, active ? 22 : 14);
  circle(b.x, b.y, active ? 22 : 14);
}

```

```

// Label beim Pinch
if (active) {
  fill(255);
  textSize(22);
  textAlign(CENTER, CENTER);
  let mx = (a.x + b.x) / 2;
  let my = (a.y + b.y) / 2 - 30;
  text(label, mx, my);
}
noStroke();
}

function drawHandSkeleton(hand) {
  let kp = hand.keypoints;

  // Verbindungen zeichnen (Fingerstruktur)
  let connections = [
    [0,1],[1,2],[2,3],[3,4],      // Daumen
    [0,5],[5,6],[6,7],[7,8],      // Zeigefinger
    [0,9],[9,10],[10,11],[11,12],  // Mittelfinger
    [0,13],[13,14],[14,15],[15,16], // Ringfinger
    [0,17],[17,18],[18,19],[19,20], // Kleiner Finger
    [5,9],[9,13],[13,17]           // Handfläche
  ];

  stroke(255, 255, 255, 50);
  strokeWeight(100);
  for (let [a, b] of connections) {
    if (kp[a] && kp[b]) line(kp[a].x, kp[a].y, kp[b].x, kp[b].y);
  }

  // Keypoints
  for (let j = 0; j < kp.length; j++) {
    // Spitzen der relevanten Finger hervorheben
    let isKey = [4, 12, 20].includes(j);
    fill(
      map(j, 0, kp.length, 0, 255),
      200,
      map(j, 0, kp.length, 255, 0),
      isKey ? 220 : 120
    );
    noStroke();
    circle(kp[j].x, kp[j].y, isKey ? 12 : 7);
  }
}

```

```
// — Eingabe & Dekodierung —————
function addSymbol(sym) {
  morseBuffer += sym;
  lastInputTime = millis();
  bgColor = color(random(255), random(255), random(255));
  playMorse(sym);
}

function decodeLetter() {
  let letter = morseReverse[morseBuffer];
  if (letter) {
    typedText += letter;
    let item = {
      code: morse[letter],
      x: width / 2,
      y: height / 2,
      targetX: random(150, width - 150),
      targetY: random(150, height - 150)
    };
    morseItems.push(item);
    for (let other of morseItems) {
      other.targetX += random(-100, 100);
      other.targetY += random(-100, 100);
      other.targetX = constrain(other.targetX, 100, width - 100);
      other.targetY = constrain(other.targetY, 100, height - 100);
    }
    bgColor = color(random(255), random(255), random(255));
  }
  morseBuffer = "";
}

// — Zeichnen —————
function drawMorse(item) {
  let spacing = 30 * morseSize;
  for (let i = 0; i < item.code.length; i++) {
    let sym = item.code[i];
    let px = item.x + (i - item.code.length / 2) * spacing;
    fill(255);
    noStroke();
    textAlign(CENTER, CENTER);
    textSize(40 * morseSize);
    if (sym === ".") text(".", px, item.y);
    if (sym === "-") text("-", px, item.y);
  }
}
```

```

function drawBuffer(progress) {
  let bx = width / 2;
  let by = height - 38;
  let barW = 220;
  noStroke();
  fill(0, 0, 0, 80);
  rect(bx - barW / 2, by + 12, barW, 6, 3);
  fill(255, 220, 0, 200);
  rect(bx - barW / 2, by + 12, barW * progress, 6, 3);
  fill(255, 240, 80);
  noStroke();
  textSize(26);
  textAlign(CENTER, CENTER);
  text("[ " + morseBuffer + " ]", bx, by);
}

function drawHUD() {
  fill(255, 255, 255, 150);
  noStroke();
  textAlign(LEFT);
  textSize(15);
  let x = 18, y = 25;
  text("👍 Daumen + kleiner Finger → · (Punkt)", x, y);
  text("👉 Daumen + Mittelfinger → - (Strich)", x, y + 22);
  text("⌚ Pause 1.5s → Buchstabe", x, y + 44);
  text("⌘ . - Space (Fallback)", x, y + 66);
}

// — Sound —————
function playMorse(code) {
  let time = 0;
  for (let sym of code) {
    let dur = sym === "." ? 120 : 350;
    setTimeout(() => {
      synth.freq(440);
      synth.amp(0.3, 0.01);
      setTimeout(() => { synth.amp(0, 0.05); }, dur);
    }, time);
    time += dur + 100;
  }
}

// — HandPose Callback —————
function gotHands(results) {
  hands = results;
}

```



```
// — Tastatur-Fallback —————
function keyTyped() {
  let k = key.toLowerCase();
  if (k === "." || k === ",") { addSymbol("."); return; }
  if (k === "-") { addSymbol("-"); return; }
  if (k === " ") { if (morseBuffer.length > 0) decodeLetter();
return; }
  if (morse[k]) {
    typedText += k;
    bgColor = color(random(255), random(255), random(255));
    let item = {
      code: morse[k],
      x: width / 2, y: height / 2,
      targetX: random(150, width - 150),
      targetY: random(150, height - 150)
    };
    morseItems.push(item);
    for (let other of morseItems) {
      other.targetX += random(-100, 100);
      other.targetY += random(-100, 100);
      other.targetX = constrain(other.targetX, 100, width - 100);
      other.targetY = constrain(other.targetY, 100, height - 100);
    }
    playMorse(morse[k]);
  }
}

function keyPressed() {
  if (keyCode === BACKSPACE) {
    if (morseBuffer.length > 0) {
      morseBuffer = morseBuffer.slice(0, -1);
    } else {
      typedText = typedText.slice(0, -1);
      if (morseItems.length > 0) morseItems.pop();
    }
  }
}
}
```

Trying out stuff with Facemesh



```
// {"P5LIVE":{"name":"facemesh","mod":1780345932632}}

let libs = ['https://unpkg.com/ml5@1/dist/ml5.js']
let faceMesh;
let video;
let faces = []
let options = { maxFaces: 1, refineLandmarks: false, flipped: true};

function preload() {
  faceMesh = ml5.faceMesh(options);
}

function setup() {
```

```

    createCanvas(windowWidth, windowHeight)
    video = createCapture(VIDEO)
    video.size(windowWidth, windowHeight)
    video.hide()

    //Startdetecting faces from the webcam video
    faceMesh.detectStart(video, gotFaces)

}

function draw() {

    // image(video, 0, 0, width, height);
    background (0)

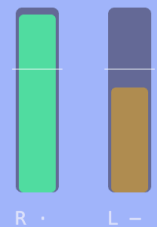
    for(let i = 0; i < faces.length; i++) {
        let face = faces[i]
        for(let j = 0; j < face.keypoints.length; j++) {
            let keypoint = face.keypoints[j];
            fill(255, 200, 105);
            noStroke();
            circle(keypoint.x, keypoint.y, 10);
        }
    }
}

//Callback function for when faceMesh outputs data
function gotFaces(results) {
    // Save the output to the faces variable
    faces = results
}

```

Mixing Face mesh with morse coding

😊 Rechte Augenbraue → · (Punkt)
😊 Linke Augenbraue → – (Strich)
🕒 Pause 1.5s → Buchstabe
= . – Space (Fallback)



ig

[·]

```
// {"P5LIVE":{"name":"new_001","mod":1780346049683}}

// {"P5LIVE":{"name":"Morse Eyebrow Control","mod":1779614887334}}
// -----
// STEUERUNG:
// 😊 Rechte Augenbraue hochziehen → · (Punkt)
// 😊 Linke Augenbraue hochziehen → – (Strich)
// 🕒 1.5 Sek. Pause → Buchstabe dekodieren
//
// Tastatur-Fallback:
// . → Punkt – → Strich Space → jetzt dekodieren
// Backspace → letztes Symbol / Buchstaben löschen
//
```

```

// Kalibrierung läuft automatisch:
// Die ersten ~3 Sekunden neutral schauen, dann beginnen.
// -----

let libs = ['https://unpkg.com/ml5@1/dist/ml5.js'];

// — Morse —————
let bgColor;
let typedText = '';
let morseItems = [];
let synth;
const morseSize = 2;

const morse = {
  a: ".-.", b: "-...", c: "-.-.", d: "-..",
  e: ".", f: "..-.", g: "--.", h: "....",
  i: "..", j: ".---", k: "-.-", l: "-..",
  m: "--", n: "-.", o: "---", p: "--.",
  q: "--.-", r: "-.-", s: "...", t: "-",
  u: "..-", v: "...-", w: ".--", x: "-.-.-",
  y: "-.-", z: "--.."
};

const morseReverse = {};
for (let l in morse) morseReverse[morse[l]] = l;

// — FaceMesh —————
let faceMesh;
let video;
let faces = [];
const faceOptions = { maxFaces: 1, refineLandmarks: false, flipped: true };

// — Gestensteuerung —————
let morseBuffer = '';
let lastInputTime = 0;
const LETTER_TIMEOUT = 1500; // ms Pause → Buchstabe dekodieren
const BROW_COOLDOWN = 800; // ms zwischen zwei Auslösungen
const RAISE_THRESHOLD = 0.09; // normierter Anstieg über Baseline → auslösen

let rightBrowTimer = 0;
let leftBrowTimer = 0;

// Laufende Baseline (Neutral-Position)
let rBaseline = null;
let lBaseline = null;
const BASELINE_ALPHA = 0.012; // wie schnell sie sich anpasst (langsam!)

```

```

// Geglättete Raise-Werte für Visualisierung
let smoothRRight = 0;
let smoothRLeft  = 0;
const SMOOTH_A    = 0.25;

// — MediaPipe Landmark-Indices —————
//
// Rechte Augenbraue (anatomisch): Mitte ~ 105
// Rechtes Auge oben (anatomisch): 159
// Rechtes Auge Ecken: außen=33, innen=133
//
// Linke Augenbraue (anatomisch): Mitte ~ 334
// Linkes Auge oben (anatomisch): 386
// Linkes Auge Ecken: innen=362, außen=263
//
// (Mit flipped:true erscheint rechts auf der linken Bildschirmseite)

// — Setup —————
function preload() {
  faceMesh = ml5.faceMesh(faceOptions);
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  bgColor = color(255, 0, 127);

  synth = new p5.Oscillator('sine');
  synth.start();
  synth.amp(0);
  textFont('monospace');

  video = createCapture(VIDEO);
  video.size(320, 240);
  video.hide();

  faceMesh.detectStart(video, gotFaces);
}

// — Draw —————
function draw() {
  background(bgColor);

  // Auto-Dekodierung nach Timeout
  if (morseBuffer.length > 0 && millis() - lastInputTime > LETTER_TIMEOUT) {
    decodeLetter();
  }
}

```

```

// Gestenanalyse
if (faces.length > 0) {
  detectEyebrows(faces[0]);
}

// Schwebende Morsezeichen
for (let item of morseItems) {
  item.targetX += random(-0.3, 0.3);
  item.targetY += random(-0.3, 0.3);

  for (let other of morseItems) {
    if (item !== other) {
      let d = dist(item.x, item.y, other.x, other.y);
      if (d < 170) {
        let angle = atan2(item.y - other.y, item.x - other.x);
        let force = (170 - d) * 0.05;
        item.targetX += cos(angle) * force;
        item.targetY += sin(angle) * force;
      }
    }
  }
  item.targetX = constrain(item.targetX, 100, width - 100);
  item.targetY = constrain(item.targetY, 100, height - 200);
  item.x = lerp(item.x, item.targetX, 0.03);
  item.y = lerp(item.y, item.targetY, 0.03);
  drawMorse(item);
}

// Getippter Text
fill(255);
noStroke();
textAlign(CENTER);
textSize(40);
text(typedText, width / 2, height - 80);

// Morsepuffer + Fortschrittsbalken
let progress = morseBuffer.length > 0
  ? map(millis() - lastInputTime, 0, LETTER_TIMEOUT, 0, 1, true) : 0;
drawBuffer(progress);

// HUD
drawHUD();
}

// — Augenbrauen-Erkennung —————

```

```

function detectEyebrows(face) {
    let kp = face.keypoints;

    // Augenbreiten zur Normierung
    let rEyeW = eyeWidth(kp, 33, 133);
    let lEyeW = eyeWidth(kp, 362, 263);

    // Normierter Abstand: Augenbraue → Augenlid (höher = Braue weiter oben)
    let rDist = browEyeDist(kp, 105, 159, rEyeW);
    let lDist = browEyeDist(kp, 334, 386, lEyeW);

    // Baseline initialisieren
    if (rDist !== null && rBaseline === null) rBaseline = rDist;
    if (lDist !== null && lBaseline === null) lBaseline = lDist;
    if (rBaseline === null || lBaseline === null) return;

    // Raise = Abstand über Baseline (positiv = Braue hochgezogen)
    let rRaise = rDist - rBaseline;
    let lRaise = lDist - lBaseline;

    // Smooth für Anzeige
    smoothRRight = lerp(smoothRRight, rRaise, SMOOTH_A);
    smoothRLeft = lerp(smoothRLeft, lRaise, SMOOTH_A);

    let now = millis();

    // Rechte Braue → Punkt (nur wenn linke Braue ruhig)
    if (rRaise > RAISE_THRESHOLD && now - rightBrowTimer > BROW_COOLDOWN) {
        if (lRaise < RAISE_THRESHOLD * 0.55) {
            addSymbol(".");
            rightBrowTimer = now;
        }
    } else if (rRaise < RAISE_THRESHOLD * 0.35) {
        // Baseline nur anpassen wenn neutral
        rBaseline = lerp(rBaseline, rDist, BASELINE_ALPHA);
    }

    // Linke Braue → Strich (nur wenn rechte Braue ruhig)
    if (lRaise > RAISE_THRESHOLD && now - leftBrowTimer > BROW_COOLDOWN) {
        if (rRaise < RAISE_THRESHOLD * 0.55) {
            addSymbol("-");
            leftBrowTimer = now;
        }
    } else if (lRaise < RAISE_THRESHOLD * 0.35) {
        lBaseline = lerp(lBaseline, lDist, BASELINE_ALPHA);
    }
}

```



```

// Visuelles Feedback
drawBrowFeedback(kp, rRaise, lRaise);
}

function eyeWidth(kp, a, b) {
  if (!kp[a] || !kp[b]) return 0;
  return dist(kp[a].x, kp[a].y, kp[b].x, kp[b].y);
}

function browEyeDist(kp, browIdx, eyeIdx, eyeW) {
  if (!kp[browIdx] || !kp[eyeIdx] || eyeW === 0) return null;
  // Differenz Y (Braue hat kleineres Y als Auge → positiv)
  return (kp[eyeIdx].y - kp[browIdx].y) / eyeW;
}

function drawBrowFeedback(kp, rRaise, lRaise) {
  let sides = [
    { browIdx: 105, raise: rRaise, label: "R → .", col: color(80, 220, 160) },
    { browIdx: 334, raise: lRaise, label: "L → -", col: color(255, 180, 60) }
  ];

  for (let s of sides) {
    if (!kp[s.browIdx]) continue;
    let x = kp[s.browIdx].x;
    let y = kp[s.browIdx].y;
    let active = s.raise > RAISE_THRESHOLD;

    // Bogen über der Augenbraue
    noFill();
    stroke(active ? s.col : color(255, 255, 255, 60));
    strokeWeight(active ? 3 : 1.5);
    arc(x, y, 50, 24, PI, TWO_PI);

    // Pfeil nach oben wenn aktiv
    if (active) {
      stroke(s.col);
      strokeWeight(2);
      line(x, y - 20, x, y - 40);
      line(x, y - 40, x - 7, y - 32);
      line(x, y - 40, x + 7, y - 32);

      fill(s.col);
      noStroke();
    }
  }
}

```

```

        textSize(14);
        textAlign(CENTER, CENTER);
        text(s.label, x, y - 55);
    }
    noStroke();
}
}

```

```

// — Eingabe & Dekodierung —————
function addSymbol(sym) {
    morseBuffer += sym;
    lastInputTime = millis();
    bgColor = color(random(255), random(255), random(255));
    playMorse(sym);
}

```

```

function decodeLetter() {
    let letter = morseReverse[morseBuffer];
    if (letter) {
        typedText += letter;
        let item = {
            code:    morse[letter],
            x:       width / 2,
            y:       height / 2,
            targetX: random(150, width - 150),
            targetY: random(150, height - 150)
        };
        morseItems.push(item);
        for (let other of morseItems) {
            other.targetX += random(-100, 100);
            other.targetY += random(-100, 100);
            other.targetX = constrain(other.targetX, 100, width - 100);
            other.targetY = constrain(other.targetY, 100, height - 100);
        }
        bgColor = color(random(255), random(255), random(255));
    }
    morseBuffer = "";
}

```

```

// — Zeichnen —————
function drawMorse(item) {
    let spacing = 30 * morseSize;
    for (let i = 0; i < item.code.length; i++) {
        let sym = item.code[i];
        let px = item.x + (i - item.code.length / 2) * spacing;
        fill(255);
    }
}

```

```

    noStroke();
    textAlign(CENTER, CENTER);
    textSize(40 * morseSize);
    if (sym === ".") text(".", px, item.y);
    if (sym === "-") text("-", px, item.y);
  }
}

function drawBuffer(progress) {
  let bx = width / 2;
  let by = height - 38;
  let barW = 220;
  noStroke();
  fill(0, 0, 0, 80);
  rect(bx - barW / 2, by + 12, barW, 6, 3);
  fill(255, 220, 0, 200);
  rect(bx - barW / 2, by + 12, barW * progress, 6, 3);
  fill(255, 240, 80);
  noStroke();
  textSize(26);
  textAlign(CENTER, CENTER);
  text("[ " + morseBuffer + " ]", bx, by);
}

function drawHUD() {
  // Kalibrierungshinweis solange keine Baseline
  if (rBaseline === null) {
    fill(255, 255, 100, 220);
    textSize(20);
    textAlign(CENTER);
    noStroke();
    text("⌚ Bitte neutral in die Kamera schauen...", width / 2, 50);
    return;
  }

  fill(255, 255, 255, 150);
  noStroke();
  textAlign(LEFT);
  textSize(15);
  let x = 18, y = 25;
  text("😬 Rechte Augenbraue → · (Punkt)", x, y);
  text("😬 Linke Augenbraue → - (Strich)", x, y + 22);
  text("🕒 Pause 1.5s → Buchstabe", x, y + 44);
  text("⌘ . - Space (Fallback)", x, y + 66);

  // Raise-Balken als Echtzeit-Feedback

```

```

    drawRaiseBar(width - 120, height / 2, smoothRRight, "R · ", color(80, 220,
160));
    drawRaiseBar(width - 60, height / 2, smoothRLeft, "L – ", color(255,
180, 60));
}

function drawRaiseBar(x, cy, raise, label, col) {
    let maxH = 120;
    let barH = constrain(map(raise, 0, RAISE_THRESHOLD * 1.5, 0, maxH), 0,
maxH);
    let triggered = raise > RAISE_THRESHOLD;

    // Hintergrund
    noStroke();
    fill(0, 0, 0, 100);
    rect(x - 14, cy - maxH / 2, 28, maxH, 4);

    // Füllstand
    fill(triggered ? col : lerpColor(col, color(100), 0.5));
    rect(x - 12, cy + maxH / 2 - barH, 24, barH, 3);

    // Schwellenlinie
    let threshY = cy + maxH / 2 - map(RAISE_THRESHOLD, 0, RAISE_THRESHOLD *
1.5, 0, maxH);
    stroke(255, 255, 255, 120);
    strokeWeight(1);
    line(x - 16, threshY, x + 16, threshY);
    noStroke();

    // Label
    fill(255, 255, 255, 160);
    textSize(12);
    textAlign(CENTER);
    text(label, x, cy + maxH / 2 + 18);
}

// — Sound —————
function playMorse(code) {
    let time = 0;
    for (let sym of code) {
        let dur = sym === "." ? 120 : 350;
        setTimeout(() => {
            synth.freq(440);
            synth.amp(0.3, 0.01);
            setTimeout(() => { synth.amp(0, 0.05); }, dur);
        }, time);
    }
}

```

```

    time += dur + 100;
  }
}

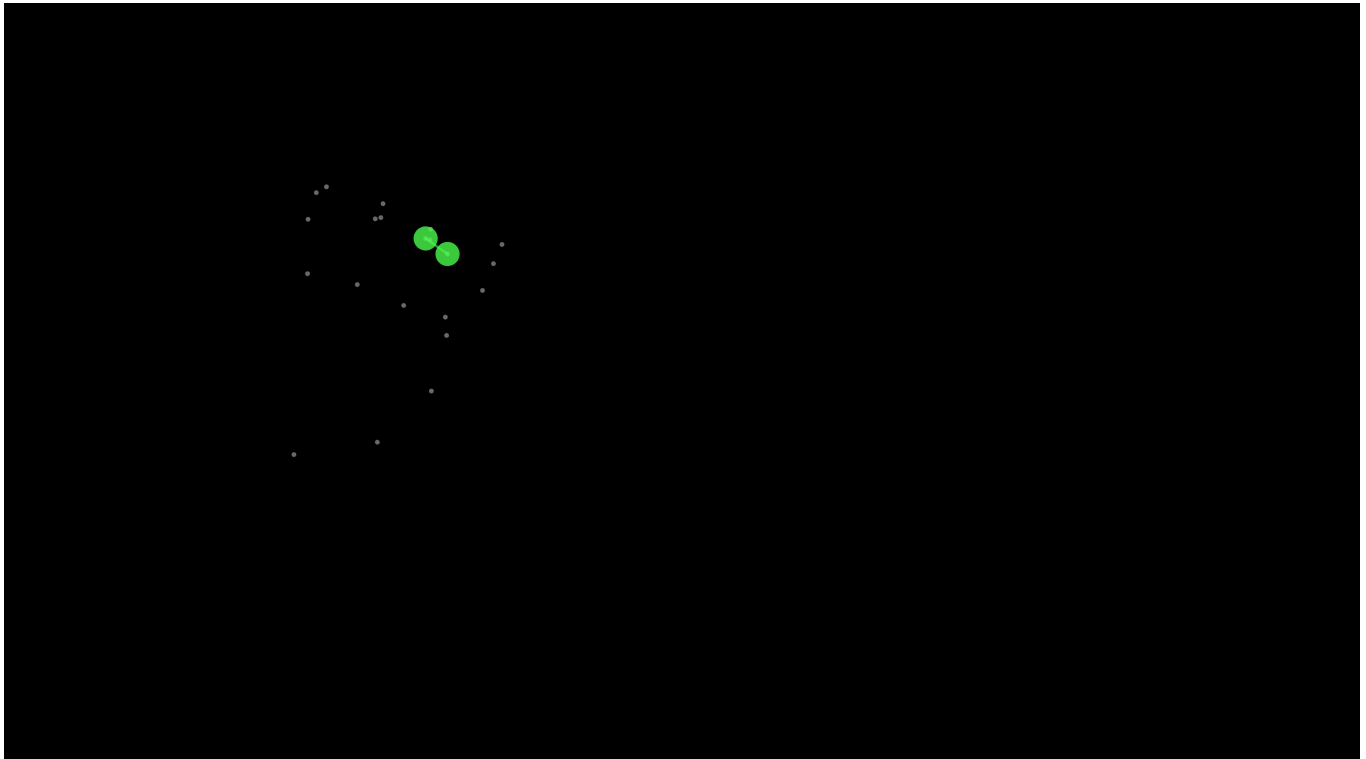
// — FaceMesh Callback —————
function gotFaces(results) {
  faces = results;
}

// — Tastatur-Fallback —————
function keyTyped() {
  let k = key.toLowerCase();
  if (k === "." || k === ",") { addSymbol("."); return; }
  if (k === "-") { addSymbol("-"); return; }
  if (k === " ") { if (morseBuffer.length > 0) decodeLetter();
return; }
  if (morse[k]) {
    typedText += k;
    bgColor = color(random(255), random(255), random(255));
    let item = {
      code: morse[k],
      x: width / 2, y: height / 2,
      targetX: random(150, width - 150),
      targetY: random(150, height - 150)
    };
    morseItems.push(item);
    for (let other of morseItems) {
      other.targetX += random(-100, 100);
      other.targetY += random(-100, 100);
      other.targetX = constrain(other.targetX, 100, width - 100);
      other.targetY = constrain(other.targetY, 100, height - 100);
    }
    playMorse(morse[k]);
  }
}

function keyPressed() {
  if (keyCode === BACKSPACE) {
    if (morseBuffer.length > 0) {
      morseBuffer = morseBuffer.slice(0, -1);
    } else {
      typedText = typedText.slice(0, -1);
      if (morseItems.length > 0) morseItems.pop();
    }
  }
}

```

Making music with my hands (this code seemed extremely buggy)



```
// {"P5LIVE":{"name":"Finger Instrument","mod":1780346358255}}

let libs = ['https://unpkg.com/ml5@1/dist/ml5.js'];

// optional background pattern
strudel $: s("bd(3,8) sd, hh*<4 8 16>").dec(.2).delay(.4) // strudel

let handPose;
let video;
let hands = [];
let options = { maxHands: 2, flipHorizontal: true};

const PINCH_DIST = 40;
const THUMB_TIP = 4;
const FINGER_TIPS = [8, 12, 16, 20]; // index, middle, ring, pinky

// note frequencies per finger
const freqs = {
  left: [220.00, 261.63, 329.63, 392.00], // A3, C4, E4, G4
  right: [440.00, 523.25, 659.25, 783.99] // A4, C5, E5, G5
};

const oscTypes = ['sine', 'triangle', 'sine', 'triangle'];

const fingerColors = [
  [255, 80, 80], // index - red
```

```

    [80, 255, 80], // middle - green
    [80, 80, 255], // ring - blue
    [255, 220, 50], // pinky - yellow
];

let pinchActive = {
  left: [false, false, false, false],
  right: [false, false, false, false]
};

let audioCtx;

function playNote(freq, type) {
  const osc = audioCtx.createOscillator();
  const gain = audioCtx.createGain();
  osc.connect(gain);
  gain.connect(audioCtx.destination);
  osc.type = type;
  osc.frequency.value = freq;
  gain.gain.setValueAtTime(0.3, audioCtx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.001, audioCtx.currentTime + 0.8);
  osc.start();
  osc.stop(audioCtx.currentTime + 0.8);
}

function mousePressed() {
  if (!audioCtx) audioCtx = new AudioContext();
}

function preload() {
  handPose = ml5.handPose(options);
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  video = createCapture(VIDEO, { flipped: true });
  video.size(width, height);
  video.hide();
  handPose.detectStart(video, gotHands);
}

function draw() {
  background(0);
  //image(video, 0, 0, width, height);

  // prompt to click if audio not started

```

```

if (!audioCtx) {
  fill(255);
  noStroke();
  textAlign(CENTER);
  textSize(20);
  text('Click anywhere to activate audio', width / 2, 40);
}

for (let i = 0; i < hands.length; i++) {
  const hand = hands[i];
  const side = hand.handedness.toLowerCase();

  // draw all keypoints
  for (let j = 0; j < hand.keypoints.length; j++) {
    const kp = hand.keypoints[j];
    fill(180, 180, 180, 150);
    noStroke();
    circle(kp.x, kp.y, 6);
  }

  const thumb = hand.keypoints[THUMB_TIP];

  for (let f = 0; f < FINGER_TIPS.length; f++) {
    const finger = hand.keypoints[FINGER_TIPS[f]];
    const d = dist(thumb.x, thumb.y, finger.x, finger.y);
    const isPinching = d < PINCH_DIST;
    const c = fingerColors[f];

    if (isPinching) {
      // visual feedback
      fill(c[0], c[1], c[2], 200);
      noStroke();
      circle(finger.x, finger.y, 30);
      circle(thumb.x, thumb.y, 30);
      stroke(c[0], c[1], c[2], 150);
      strokeWeight(3);
      line(finger.x, finger.y, thumb.x, thumb.y);
      noStroke();

      // trigger sound only on first frame of pinch
      if (!pinchActive[side][f] && audioCtx) {
        playNote(freqs[side][f], oscTypes[f]);
        pinchActive[side][f] = true;
      }
    } else {
      pinchActive[side][f] = false;
    }
  }
}

```



```
    }  
  }  
}
```

```
function gotHands(results) {  
  hands = results;  
}
```